

学習管理アプリ開発仕様書

インサイトページ拡張機能：仕様 & ローカル実装ロジック詳細

1. 基本設計方針とコアロジック

本機能は、外部サーバーへの依存（APIキー設定、通信量制限、プライバシーリスク）を完全に排除し、**フロントエンドのJavaScript環境のみで完結する超軽量・高セキュリティな自己分析インサイト機能**である。特に不規則になりがちな学習者の生活リズムを定量的に可視化し、客観的な自己改善行動を促すためのロジックを規定する。

1.1 基準時間定義（医学生・夜型生活リズムへの最適化）

一般的な日付切り替え時刻（00:00）ではなく、深夜・早朝の学習実態に即して「**午前 05:00**」を日付の境界線とする。

- 論理日付 (Logical Date):** 午前05:00から翌午前04:59までを同一の1日として処理する。
- getLogicalDate() ロジック:** 対象タイムスタンプから5時間を減算した時点の日付をそのログの論理日付とする。

1.2 分析対象期間の定義

期間区分	相対定義 (基準: logicalToday)	目的
今週 (直近7日間)	0日前 ~ 6日前 までの計7日間	現在のリアルタイムな生活リズムと学習量の把握
先週 (その前の7日間)	7日前 ~ 13日前 までの計7日間	直近の変化を検知するための比較対象 (対照群)
長期傾向 (直近30日間)	0日前 ~ 29日前 までの計30日間	クロノタイプ・学習ペース・環境相関などのパーソナリティ判定

2. 各分析項目の算出アルゴリズムとロジック

A. 最近の生活リズム & 傾向の変化（前週比較）

A-1. 平均勉強開始時刻

計算目的: 朝型の維持、あるいは夜型化・生活リズムの後退（崩れ）を検知する。

【重要】深夜帯をまたぐ平均値計算のバグ回避ロジック

単純な時刻の分換算（0時=0分～23時=1399分）で平均を行うと、23:00開始（1380分）と深夜1:00開始（60分）の平均が12:00（720分）になってしまう。これを防ぐため、「朝5:00を起点（0分）」とした24時間スケールで計算を行う。

1. 各論理日における「最も早い勉強ログ（started_atが朝5:00以降）」を抽出。
2. 開始時刻を「5:00からの経過分数」に変換。
 - 式: $MinutesFromBase = (Hour \times 60 + Minute) - 300$
 - ※ 結果がマイナス（0:00～4:59の間）になる場合は + 1440 分する。
3. 期間内の経過分数の算術平均を算出。
4. 平均分数を通常の「時:分」形式に逆変換（300分を足し、1440分を超えたら1440を引く）。

A-2. 1日平均学習時間

計算ロジック: 期間中の総勉強時間（分）を7（日間）で除算し、時間（h）に変換。前週比増減率を算出。

- 増減率（%）式: $Change\% = \left(\frac{ext{今週の平均} - ext{先週の平均}}{ext{先週の平均}} \right) \times 100$

A-3. 平均集中度

計算ロジック: 期間内に記録された全ログの集中度（★1～5）の単純平均。前週比の差分（例: +0.2）を算出。

A-4. 深夜学習の警戒アラート

計算ロジック: 深夜帯（23:00 ～ 翌05:00）に開始された勉強時間の合計が、全体の総勉強時間に占める割合（%）を今週・先週でそれぞれ算出。今週の割合が先週より **5%以上 増加** している場合に警告バッジを付与。

B. パーソナル自己分析（直近30日の長期傾向）

B-1. 時間タイプ診断（クロノタイプ判定）

直近30日間の総勉強時間における、時間帯別の勉強量割合から以下のように自動判定する。

- 朝型スプリンター 🌅: 朝（05:00～11:00）の勉強時間が全体の **40%以上** を占める場合。
- 夜型ディープフォーカス 🌙: 夜・深夜（17:00～翌05:00）の勉強時間が全体の **50%以上** を占める場合。
- オールラウンダー ⚖️: 上記のいずれの条件にも偏らず、バランスよく分散している場合。

B-2. 学習ペースタイプ診断

直近30日間の「日別勉強時間」のばらつき度合い（変動係数: **CV**）から、日々の継続性を判定する。

数式定義: 変動係数 $CV = \frac{\sigma}{\mu}$ （ここで σ は日別勉強時間の標準偏差、 μ は日別勉強時間の平均値）

- コツコツ習慣化タイプ 🧑: $CV < 0.6$ の場合。毎日安定して同じくらいの時間を確保できている。

- ・週末・追い込み集中タイプ ⚡: $CV \setminus ge 0.6$ の場合。やる日とやらない日の差が激しく、特定日に一気に集中している。

B-3. ベスト集中環境の特定

location (自宅・図書館・カフェ等) と timeSlot (朝・昼・夜・深夜) の組み合わせ (複合キー) ごとにログをグループ化。データの信頼性を保証するため、「ログ数が3件以上」の組み合わせに限定し、最も「平均集中度 (★)」が高かったトップ1の組み合わせを抽出・表示する。

C. 生活リズム・睡眠関連拡張ロジック (追加要件)

起床時刻 (wake_up_time) および就寝時刻 (bedtime) を日別にローカル保存するデータモデルに基づき、以下の3インサイトを生成する。

C-1. 起床時刻の安定度 (リズムのブレ)

直近7日間の起床時刻のばらつき (標準偏差) を算出。起床時刻を「朝5:00からの経過分数」に変換して計算。

- 良好: 標準偏差が30分未満 (極めて規則正しい)
- 警戒: 標準偏差が30分以上60分未満 (やや乱れ気味)
- 要注意: 標準偏差が60分以上 (日によって起床が大きくズレている)

C-2. 初動タイムラグ (起床から勉強開始まで)

各論理日において、「その日の最初の勉強ログの started_at」 - 「その日の wake_up_time」 を分換算で算出。今週の平均タイムラグを求め、先週の平均と比較。朝の行動の迅速性を可視化する。

C-3. ベスト睡眠時間の特定 (睡眠時間×翌日集中度相関)

「翌論理日の起床時刻」 - 「当論理日の就寝時刻」 で睡眠時間を算出。過去30日間のログから、睡眠時間を4つのスロット (①6h未満、②6~7h、③7~8h、④8h以上) に分類。それぞれの睡眠時間を取った「翌日」の学習ログの【平均集中度】を計算し、最も数値が高いスロットを「あなたのベスト睡眠時間」として抽出する。

C-4. 就寝前クールダウン分析

当日の「最後の学習ログの終了時刻 (ended_at)」から「就寝時刻」までの時間 (分) を算出。これが30分未満の日が直近7日間で3日以上ある場合、「⚠️ 就寝直前まで勉強する傾向があり、睡眠の質を下げている可能性があります」というアラートを表示。

C-5. 夜更かしバッファ超過アラート (夜型化の早期検知)

過去30日間の平均就寝時刻を基準とし、直近3日間連続して、平均より90分以上遅い就寝が記録された場合、自動的に「⚠️ 要注意: 3日連続で就寝が深く後退しています。夜型化の兆候です」と警告カードを最上位にスタックさせる。

3. フロントエンドJavaScript実装用コード雛形

以下は、外部ライブラリを一切使わずに純粋なJavaScriptのみで「変動係数(CV)」および「5:00起点分換算」を処理するための主要コードスニペットである。

```
/**
 * 学習ペースの変動係数(CV)を計算する関数
 * @param {number[]} dailyHours - 直近30日間の日別勉強時間配列
 * @returns {number} 変動係数(CV)
 */
function calculateLearningPaceCV(dailyHours) {
  if (!dailyHours || dailyHours.length === 0) return 0;

  const count = dailyHours.length;
  const sum = dailyHours.reduce((acc, val) => acc + val, 0);
  const mean = sum / count;

  if (mean === 0) return 0; // 勉強時間がゼロの場合は0を返す

  // 分散の計算
  const variance = dailyHours.reduce((acc, val) => acc + Math.pow(val - mean, 2), 0) / count;
  // 標準偏差
  const standardDeviation = Math.sqrt(variance);

  // 変動係数 (CV)
  return standardDeviation / mean;
}

/**
 * 05:00を起点とした経過分数の計算（深夜のまたぎ処理用）
 * @param {string} timeStr - "HH:MM" 形式の文字列
 * @returns {number} 05:00からの経過分数 (0 ~ 1439)
 */
function getMinutesFromBase5AM(timeStr) {
  const [hours, minutes] = timeStr.split(':').map(Number);
  let totalMinutes = hours * 60 + minutes;

  // 05:00を基準点(0分)とするためのオフセット処理
  let offsetMinutes = totalMinutes - 300;
  if (offsetMinutes < 0) {
    offsetMinutes += 1440; // 深夜0:00~4:59は、24時間分(1440分)進める
  }
  return offsetMinutes;
}
```

4. UI/UX 画面配置およびデザイン方針

4.1 配置コンポーネント構成

インサイトページ内の最上位（既存の「AI学習アドバイザー」の上部）に、横並びまたは2段構成で以下の新規カードセクションを挿入する。

1. 「生活リズムと最近の傾向」カード: 前週比較の4指標、および起床・就寝アラートなどの動的ステータスを表示。
2. 「学習タイプ自己分析」カード: クロノタイプ、習慣化CV指標、ベスト環境関連の3つの静的（直近30日）ステータスを表示。

4.2 視覚的フィードバック仕様

- **ステータスバッジ:**  **良好** (リズム・量ともに維持) /  **警戒** (前週比で開始時刻が45分以上後退、または深夜割合増加) /  **要注意** (3日連続夜更かし、または起床標準偏差60分以上) の3色をカラーコード（CSS変数）で制御する。
- **数値表現:** ポジティブな変化（学習時間+15%、集中度+0.3など）は #166534 (緑系)、ネガティブまたは警戒すべき変化（開始時刻-60分、深夜割合+8%など）は #b91c1c (赤系) もしくは #d97706 (オレンジ系) のテキストカラーで明示する。

5. 今回のスコープから除外された項目

-  **科目バランスの偏り推移:** 特定科目に偏っているかどうかのダッシュボード集計は、画面の複雑化を避けるため今回は完全に対象外とする。
-  **外部AIによるテキスト文章生成:** APIキーの入力やコール数の制限、通信遅延を考慮し、ルールベースの固定条件分岐テキスト（JavaScriptのテンプレートリテラル）のみで全メッセージを出力する。